Итоговые примеры

Поздравляю! Вы продвинулись ещё на шаг к своей цели. Чтобы закрепить пройденный материал, давайте быстренько повторим теорию и выполним 2 практических задания.

Оглавление:

1. Теория
2. Задание №1
3. Задание №2

Теория

Операторы if позволяют выполнить код, основываясь на результате условия (истино оно или нет). Если условие ложное, то выполняется **оператор else**. Можно связывать несколько операторов if и else вместе.

Оператор switch обеспечивает более удобный и быстрый способ использования условий/вложений в else. Он отлично сочетается с перечислениями.

Оператор goto позволяет перенести точку выполнения в программе из одного места в другое. Использовать этот оператор не рекомендуется.

Цикл while выполняет определенный код до тех пор, пока условие истинно. Сначала обрабатывается условие, а затем выполняется код.

Цикл do while — это тот же цикл while, только сначала выполняется код, а затем уже проверяется условие. Он отлично подходит для вывода меню или других элементов, так как позволяет выполнить код хотя бы один раз.

Циклы for наиболее используемые циклы. Они идеальны, когда нужно выполнить код определенное количество раз.

Оператор break немедленно завершить выполнение оператора switch, цикла do, while или for.

Оператор continue позволяет немедленно перейти к следующей итерации цикла. Будьте осторожны при использовании этого оператора в связке с циклом while или do while — не забывайте о возможновении проблемы с инкрементом счетчика цикла.

И, наконец, **рандомные числа** позволяют получить разные результаты при выполнении одной и той же программы.

Тест

Задание №1

В **итоговом тесте №2** мы написали программу имитации начала, подающего с базыи. Так как тогда мы ещё ничего не знали о циклах и на рекоме не использовать, то время полета мячика составляло всего лишь 5 секунд.

Измените программу, проведенную ниже, таким образом, чтобы мячик падал равно то количество секунд, которое необходимо ему для достижения земли.

constants.h:

```
#ifndef CONSTANTS_H
#define CONSTANTS_H

namespace myConstants
{
    const double gravity(9.8);
}

#endif
```

Основной файл:

```
#include "constants.h"
#include "iostream"
#include "cmath" // для использования в calculate()
double getInitialHeight()
{
    std::cout << "Enter the initial height of the tower in meters: ";
    double initialHeight;
    std::cin >> initialHeight;
    return initialHeight;
}

// Функция calculate() принимает два параметра, возвращает double
double calculate(double initialHeight, int seconds)
{
    // Формула расчета: h = h0 + v0*t + g*t^2 / 2, где v0=0 (начальная скорость)
    double distanceFallen = (myConstants::gravity * seconds * seconds) / 2;
    double currentHeight = initialHeight - distanceFallen;

    return currentHeight;
}

// Функция printHeight() принимает два параметра, ничего не возвращает
void printHeight(double height, int seconds)
{
    if (height > 0.0)
    {
        std::cout << "At " << seconds << " seconds, the ball is at height:\t" << height << " meters\n";
    }
    else
    {
        std::cout << "At " << seconds << " seconds, the ball is on the ground.\n";
    }
}

void calculateAndPrintHeight(double initialHeight, int seconds)
{
    double height = calculateHeight(initialHeight, seconds);
    printHeight(height, seconds);
}

int main()
{
    const double initialHeight = getInitialHeight();

    calculateAndPrintHeight(initialHeight, 0);
    calculateAndPrintHeight(initialHeight, 1);
    calculateAndPrintHeight(initialHeight, 2);
    calculateAndPrintHeight(initialHeight, 3);
    calculateAndPrintHeight(initialHeight, 4);
    calculateAndPrintHeight(initialHeight, 5);

    return 0;
}
```

Ответ №1

constants.h:

```
#ifndef CONSTANTS_H
#define CONSTANTS_H

namespace myConstants
{
    const double gravity(9.8);
}

#endif
```

Основной файл:

```
#include "constants.h"
#include "iostream"
#include "cmath" // для использования в calculate()
double getInitialHeight()
{
    std::cout << "Enter the initial height of the tower in meters: ";
    double initialHeight;
    std::cin >> initialHeight;
    return initialHeight;
}

// Функция calculate() принимает два параметра, возвращает double
double calculate(double initialHeight, int seconds)
{
    // Формула расчета: h = h0 + v0*t + g*t^2 / 2, где v0=0 (начальная скорость)
    double distanceFallen = (myConstants::gravity * seconds * seconds) / 2;
    double currentHeight = initialHeight - distanceFallen;

    return currentHeight;
}

// Функция printHeight() принимает два параметра, ничего не возвращает
void printHeight(double height, int seconds)
{
    if (height > 0.0)
    {
        std::cout << "At " << seconds << " seconds, the ball is at height: " << height << " meters\n";
    }
    else
    {
        std::cout << "At " << seconds << " seconds, the ball is on the ground.\n";
    }
}

int main()
{
    const double initialHeight = getInitialHeight();

    int seconds = 0;
    double height;

    while
    {
        height = calculateHeight(initialHeight, seconds);
        printHeight(height, seconds);
        ++seconds;
    } while (height > 0.0);

    return 0;
}
```

Задание №2

Напишите программу игру типа Hi-Lo:

- По первым, вам программа должна выбрать случайное целое число в диапазоне от 1 до 100.
- Пользователю дается 7 попыток, чтобы угадать это число.
- Если пользователь не угадал число, то программа должна подсказывать, была ли его догадка слишком большой или слишком маленькой.
- Если пользователь угадал число, то программа должна сообщить, что всё верно — вы выиграли.
- Если же у пользователя кончились попытки, и он не угадал число, то программа должна сообщить ему, что он проиграл и показать правильный результат.
- В конце игры программа должна спросить у пользователя, не хочет ли он сыграть еще раз. Если пользователь не ввел: **n** , **no** и (а что-то другое), то программа должна спросить его еще раз.

Пример результата выполнения вашей программы:

```
Let's play a game. I'm thinking of a number. You have 7 tries to guess what it is.
Guess #1: 64
Your guess is too high.
Guess #2: 32
Your guess is too low.
Guess #3: 54
Your guess is too high.
Guess #4: 50
Correct! You win!
Would you like to play again (y/n)? y
Let's play a game. I'm thinking of a number. You have 7 tries to guess what it is.
Guess #1: 64
Your guess is too high.
Guess #2: 32
Your guess is too low.
Guess #3: 48
Your guess is too high.
Guess #4: 36
Your guess is too high.
Guess #5: 30
Your guess is too low.
Guess #6: 45
Your guess is too low.
Guess #7: 48
Your guess is too low.
Sorry, you lose. The correct number was 49.
Would you like to play again (y/n)? n
Would you like to play again (y/n)? f
Would you like to play again (y/n)? n
Thank you for playing.
```

> **Подсказки:**
>
> - Используйте в качестве стартового числа из врамя генерации случайных чисел вызов функции time(0) .
> - **Пользователям Visual Studio:** Это та ложной реализации функции rand() (первое рандомное число не сильно отличается от стартового) — вызовите rand() сразу после установки стартового числа, чтобы сбросить первый результат.
> - Используйте функции getRandomNumber() для генерации случайного числа.
> - В функции, которая будет спрашивать у пользователя, не хочет ли он сыграть еще раз, используйте механизм обработки некорректного пользовательского ввода.

Ответ №2

```
#include "iostream" // для функции std::cin и std::cout
#include "cstdlib" // для функции srand() и rand()
#include "ctime" // для функции time()

// Генерирует случайное целое число между min и max.
// Предполагается, что srand() уже вызывался
int getRandomNumber(int min, int max)
{
    // static used for efficiency, so we only calculate this value once
    // For more info, see: https://www.learncpp.com/cpp-tutorial/6-15a-global-variables-part-one/
    static const double fraction = 1.0 / (static_cast<double>(RAND_MAX) + 1.0);
    // static_cast<double>(rand()) to fix Visual Studio bug
    return static_cast<int>(rand() * fraction * (max - min + 1) + min);
}

// Функция ask_user() спрашивает пользователя, хочет ли он сыграть еще раз.
bool playAgain(int guesses, int number)
{
    for (int count = 1; count <= guesses; ++count)
    {
        std::cout << "Guess #" << count << ": ";
        int guess;
        std::cin >> guess;

        if (guess == number)
            std::cout << "Your guess is too high.\n";
        else if (guess > number)
            std::cout << "Your guess is too low.\n";
        else // guess == number
            return true;
    }

    return false;
}

bool ask_user()
{
    // Спрашиваем пользователя, хочет ли он сыграть еще раз.
    // Если пользователь ввел 'y' или 'n', то возвращаем true, если ввел что-то еще, то спрашиваем еще раз.
    char ch;
    while
    {
        std::cout << "Would you like to play again (y/n)? ";
        std::cin >> ch;
    } while (ch != 'y' && ch != 'n');

    return (ch == 'y');
}

int main()
{
    srand(static_cast<unsigned int>(time(0)); // static_cast нужен для работы с Visual Studio
    rand(); // Иногда первое случайное число при вызове rand() не очень хорошее, поэтому вызовем rand() сейчас, чтобы сбросить первый результат

    const int guesses = 7; // количество попыток в догадке

    do
    {
        int number = getRandomNumber(1, 100); // число, которое пользователь должен угадать

        std::cout << "Let's play a game. I'm thinking of a number. You have " << guesses << " tries to guess what it is.\n";

        int const bool playAgain(guesses, number);
        if (win)
            std::cout << "Correct! You win!\n";
        else // not win
            std::cout << "Sorry, you lose. The correct number was " << number << ".\n";

    } while (ask_user());

    std::cout << "Thank you for playing.\n";
    return 0;
}
```